Lab tutorial: How Module works in JS :

CN5006 Web and Mobile Development

Dr.Nadeem Qazi

---

1. **Lab Session**
    1. Functions in JavaScript:
        a. What are JS functions
        b. How to write functions in JavaScriptt
        c. How to export a function with arguments/parameters.
        d. Defining an Arrow function

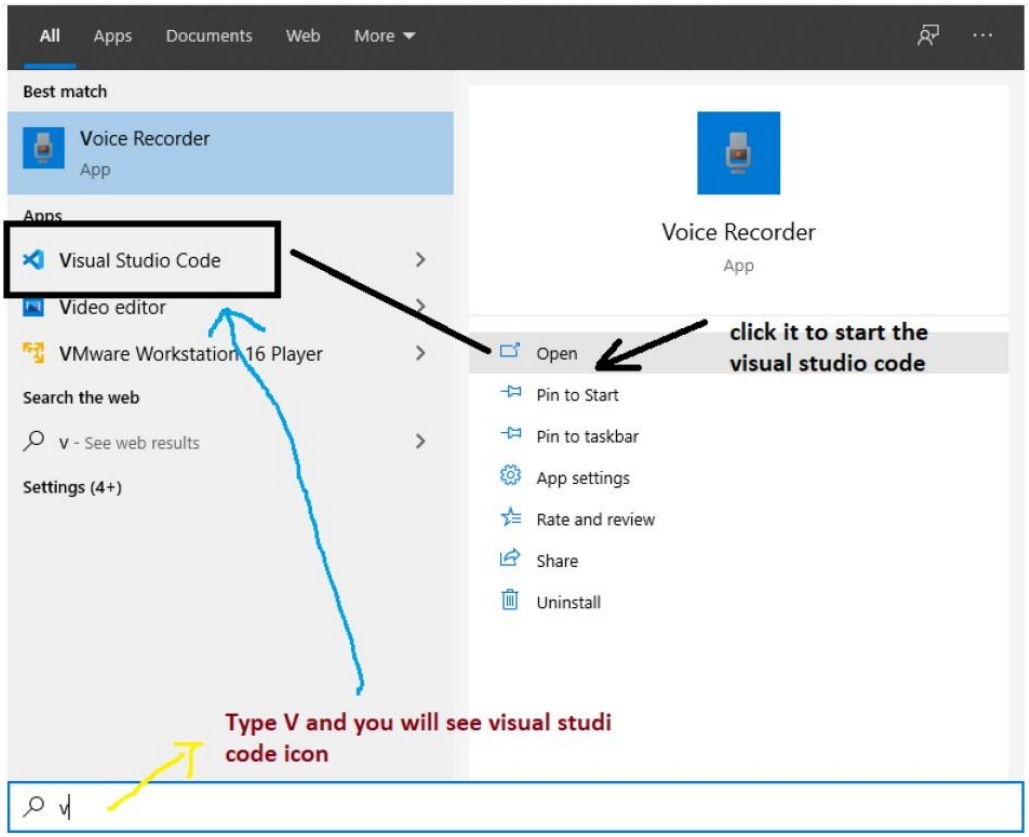
2. How to Create a Local Module in NodeJS
    - How to export a function/s and variable/s to other module
    - How to use a module declared in a separate file
3. Access NodeJS Core Module

before you write the code, please note // is used to comment in JS and it is NOT treated as CODE

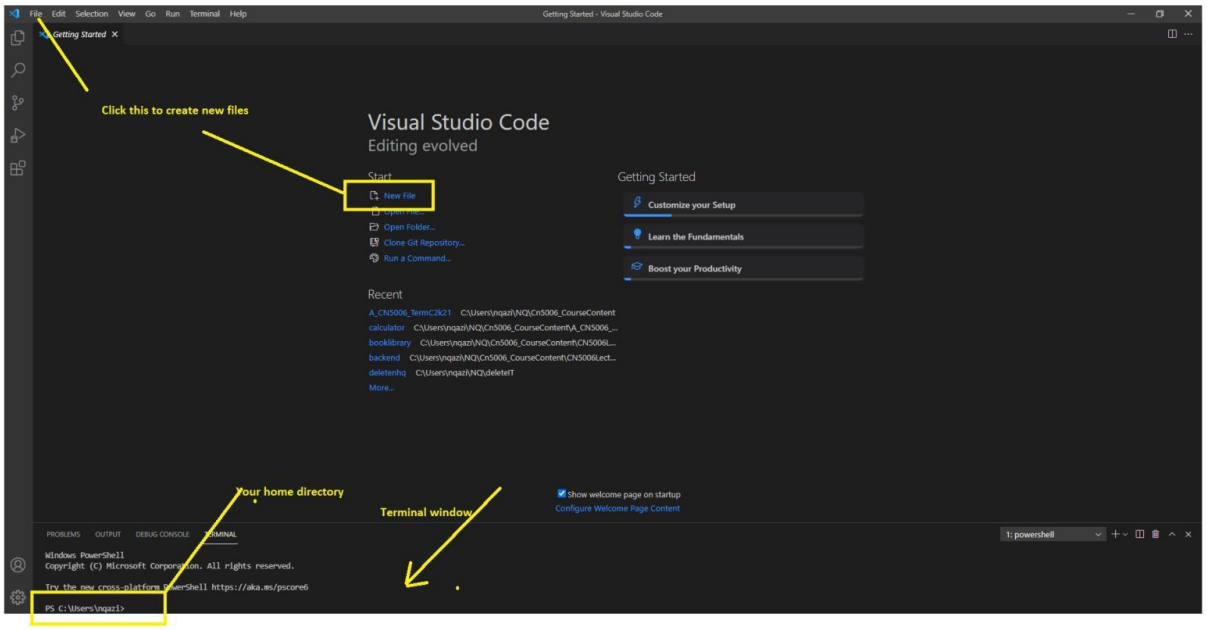
Step 1: Make sure you have installed Visual Studio Code and NodeJS.

Step 2: Open Visual Studio Code by typing V as shown in the figure

below

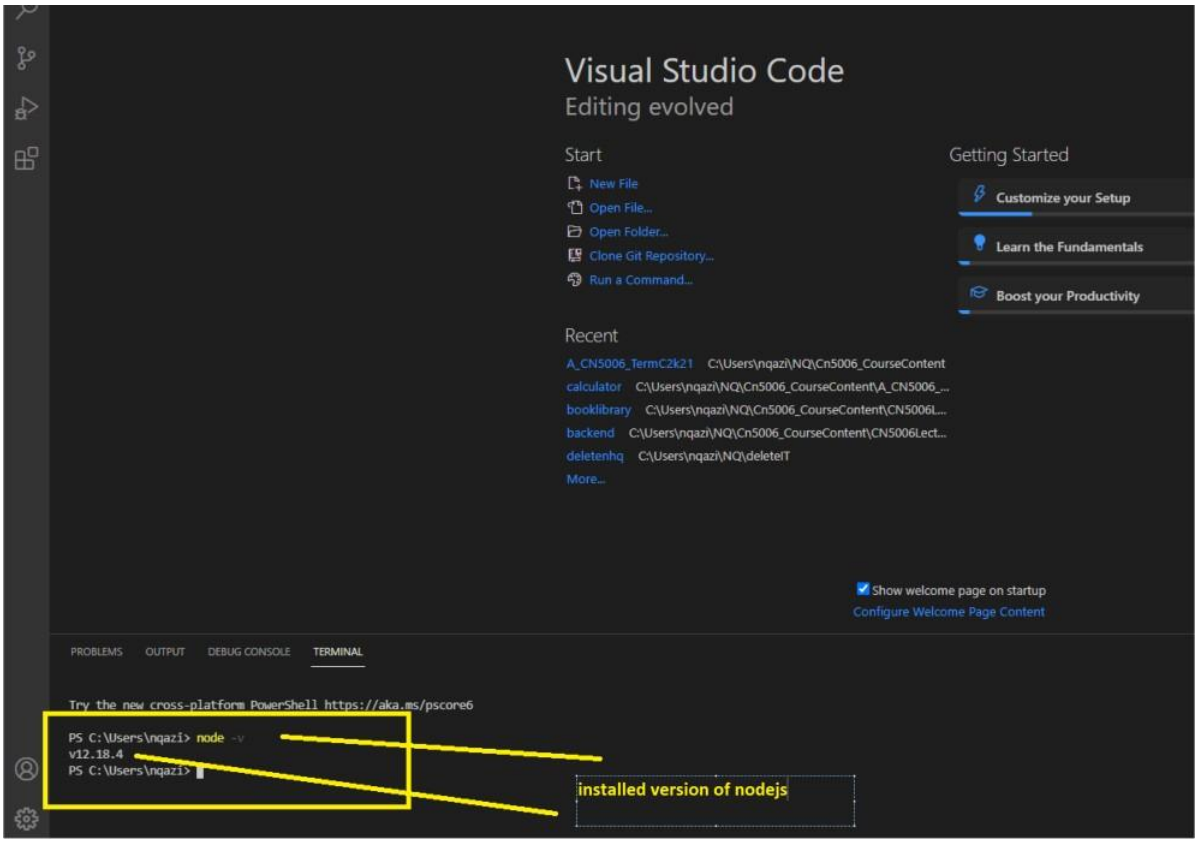


Once the visual studio code is started you will see the following window :

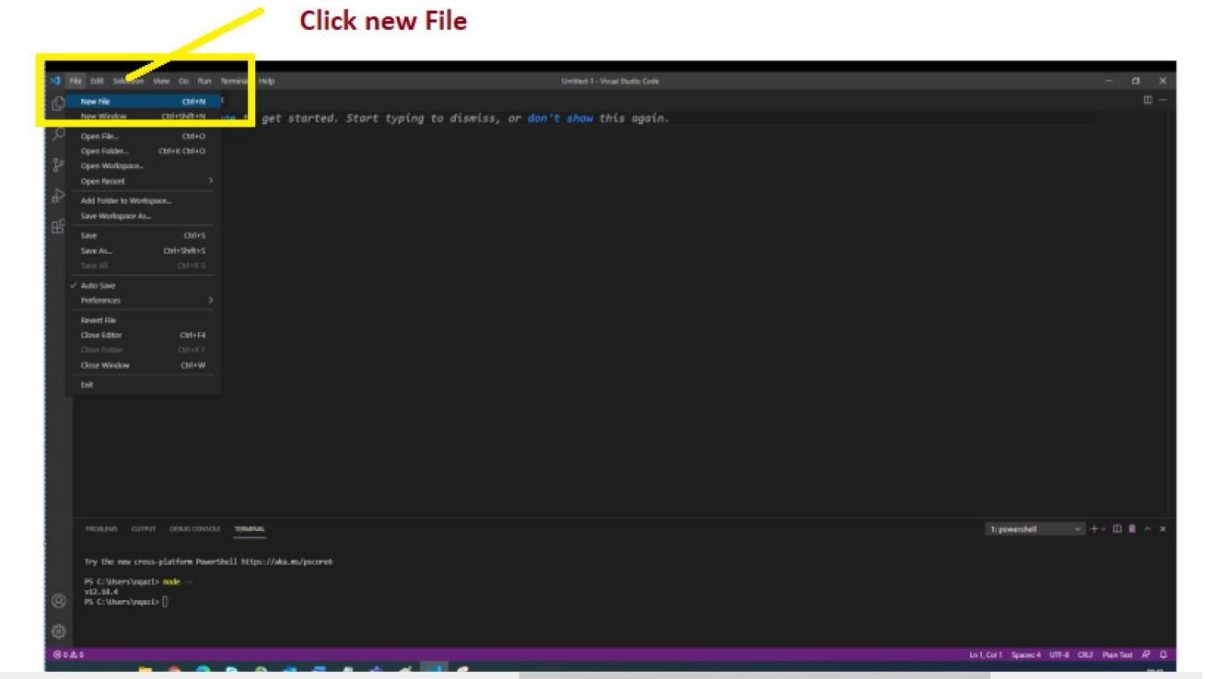


Step 3 check if node JS is installed. In the terminal window , on the prompt write following line :**node -v**

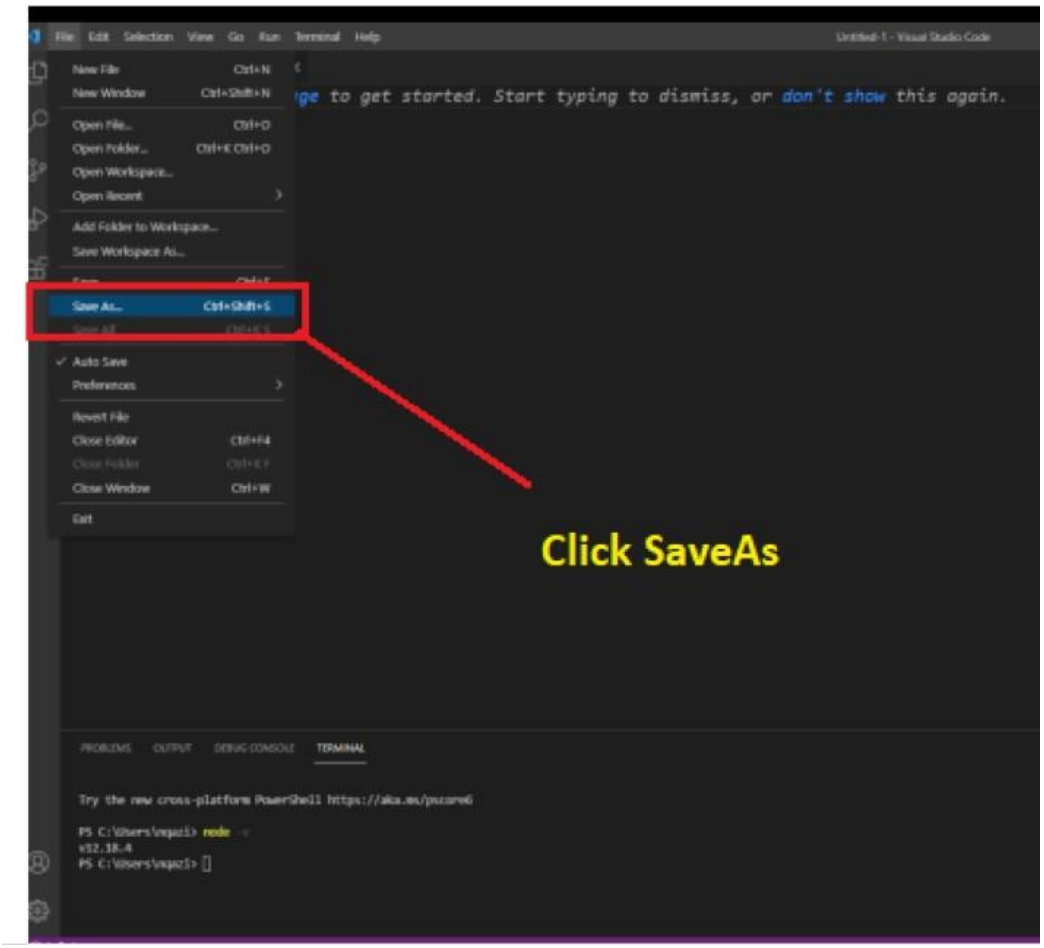
you should see the version number of the nodeJS running on your system as shown in the figure below, however you may have different version of nodeJS running



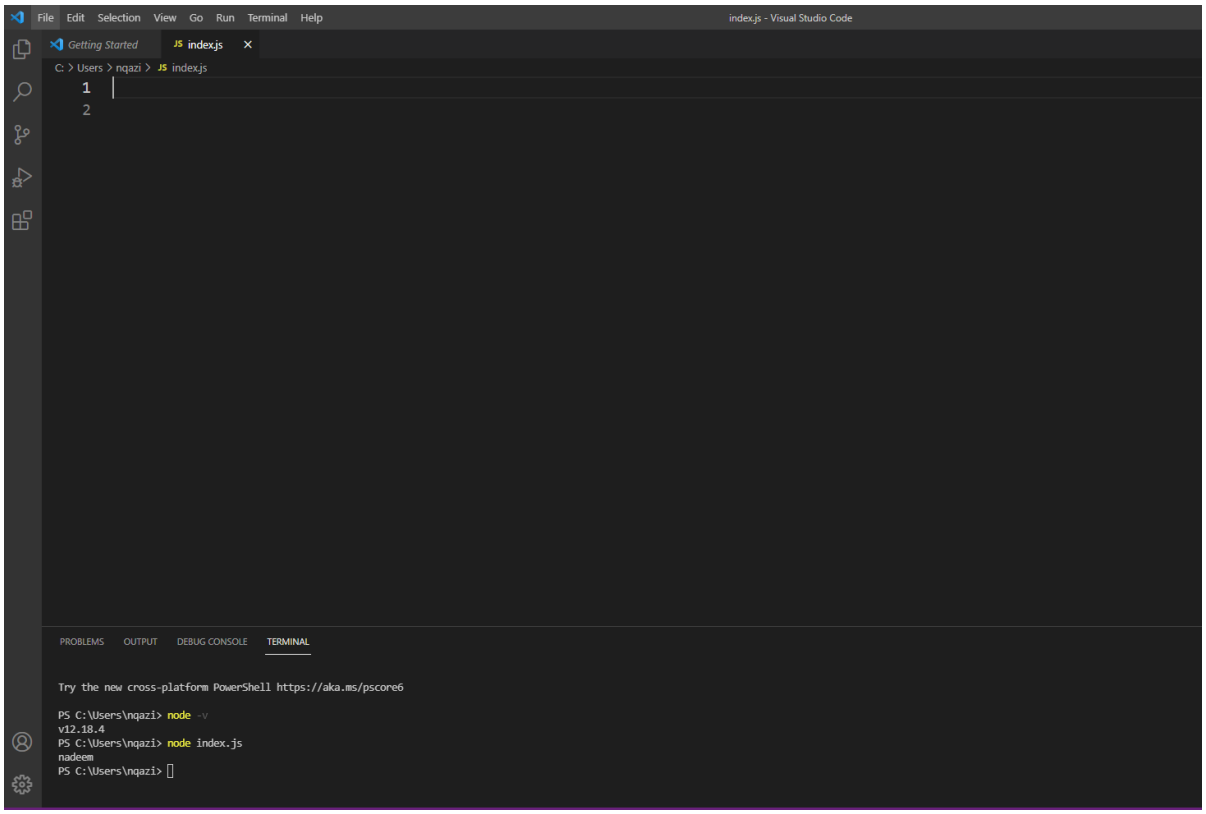
Now you are all set to start coding in Visual Studio Code. Click File ,new to create a new file as shown in the figure below



It will open a new blank file , save this file as Java script file by clicking SaveAS



It will open a dialogue box Choose the name of the file as index.js which will then create a index.js file for you to start coding . as shown in the figure below



Now Star writing the code

Exercise 1:

Exercise 1 requires you to write the code in a file name it index.js

Follow these steps to write and save a JavaScript file (index.js) that defines a function called **`Employeeinfo.js`** and calls this function with different parameter values.

- **Create an Index.js File:** In your editor, create a new file and name it `index.js`.
- **Write the Code:**
  - Inside the `index.js` file, define a function named `Employeeinfo` that accepts two parameters: `name` and `salary`.
  - Call this function with different values for the parameters. The code is shown below:, write the code and save the index.js file
  -

```
function EmployeeInfo(name, Salary)
{
console.log("Welcome " + name + " Your monthly Salary is " + Salary);
}

// Display current date and time
const currentDateTime = new Date();

```

```
console.log("This is my first program");
console.log("Programe run on : " +  currentDateTime.toLocaleString());

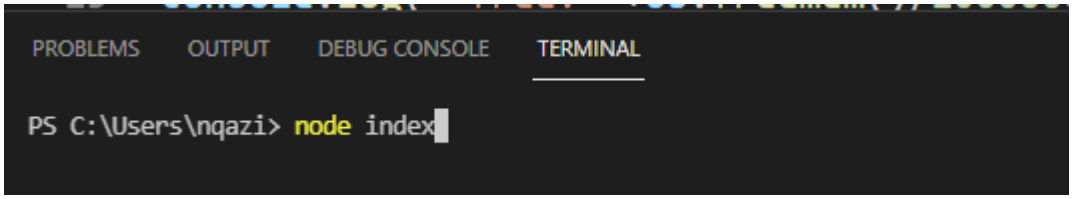
var EmpName = "John";
var EmpSalary = 50000;

// Calling the function
EmployeeInfo(EmpName, EmpSalary);
```

Save this code in index.js

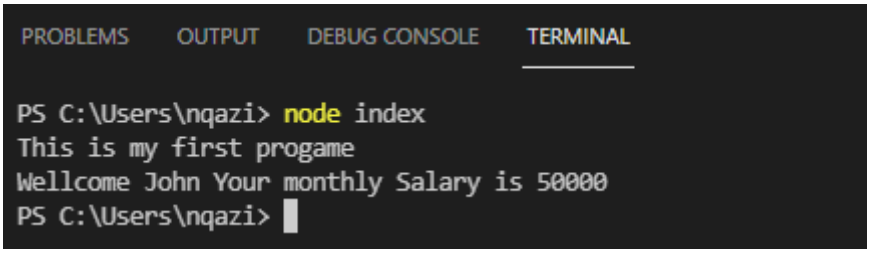
How to run this Code:

In the terminal window type " node index" and hit enter. However make sure that your terminal window should be running cmd not power shell.





You will see you code will be executed and the following message will appear in the Terminal window.



Code Explanation

1. **console.log** is used to display a message or variable value in the terminal window, so when you use in your code :
2. You define the function in Java script with the keyword function followed by name of the function which in this case is EmployeeInfo , (you can give any name to the function) .
3. The parameter/s of the function is defined in parentheses () which in this case, is (name,Salary), so the complete signature of the function is :

```
function EmployeeInfo(name,Salary)
{
    console.log("Wellcome " + name+ "Your monthly Sa
lary is "+ Salary)
}
```

Observe the use of “+” in console.log to concatenate message with the value of parameters passed in this function i.e. name and Salary.

Exercise 2 : Create Arrow Functions:

Arrow functions are a modern way of defining the functions in Javascript, Unlike traditional JavaScript functions, they do not require a name and are more concise. To learn more for watch the Arrow functions video in the lecture uploaded on Moodle.

Syntax of the ArrowFunction:

const variablename= () =>{}

Like before the parameters are passed in (), Note function keyword is NOT used .

In your index.js file that you created for exercise 1, please Add the following code :

```
// Add this for Second Exercise,Note three parameters are passed
const EmpSkills= (skills,Name,LabGrp)=> {
  console.log(" Student ID"+ Name+ "Expert in "+ skills)
  console.log("Lab Group Name",LabGrp)
}
```

Then call this arrow function with the parameter “java” , writing following line of code : EmpSkills(“Java”) in

```
EmpSkills("java","Replace this with your Student ID ","Replace with your Lab Grp")
```

index.js

The overall index.js should look like this :

```
function EmployeeInfo(name, Salary)
{
console.log("Welcome " + name + " Your monthly Salary is " + Salary);
}

// Display current date and time
const currentDateTime = new Date();

console.log("This is my first program");
console.log("Programe run on : " +  currentDateTime.toLocaleString());

var EmpName = "John";
var EmpSalary = 50000;

// Calling the function
EmployeeInfo(EmpName, EmpSalary);
// Add this for Second Exercise
const EmpSkills= (skills,Name,LabGrp)=> {
  console.log(" Student ID"+ Name+ "Expert in "+ skills)
  console.log("Lab Group Name",LabGrp)
}
EmpSkills("java","Replace this with your Student ID ","Replace with your Lab Grp")
```

Save the index.js and run it using node index.js, you will see the following output:

```
PS C:\Users\nqazi> node index
This is my first progame
Wellcome John Your monthly Salary is 50000
Expert in java
PS C:\Users\nqazi>
```

Exercise3 : Creating Local Module in NodeJS

What are Java Script Modules:

A module in Node.js is simply a JavaScript file that contains the definitions of one or more functions, objects, or variables. You can create your own modules and easily include them in your applications. Modules that are defined in separate files can be imported into another JavaScript file using the require keyword in Node.js.

Follow these steps:

For this exercise, you need three JS files.

- **StudentInfo.js (you will be creating this as a separate file)**
- **Person.js (You will be creating it as a separate js file)**
- **Index.js (you already have created this file in Exercises 1& 2)**

Follow these steps:

Create a separate Java script file(click File ➔new) and save it as StudentInfo. js and write following code in this(Studentinfo.js) file:

FileName: StudentInfo.js

// arrow function with no name, this function does not take any argument vrbname = () => {}
// code starts from here:

```
const dateofBirth= "12/12/1980"

const getStudentName =    ()  =>
{
return "write your name here"
}
const getCampusName = () =>
{return ("UEL Campus ")
}
//exporting functions & variable outside the module
exports.getName=getStudentName
exports.Location=getCampusName
exports.dob=dateofBirth
// How  to export function with parameters
exports.Studentgrade=(marks)=>
        {
        if (marks>50 && marks <70) return ("B grade")
        else
            return ("A grade)")
        }
```

Step 2 : Create Another file name this file person.js.This is your another local module and type following code into it

Person.js

when you have a module that exports just one thing, it's more common to use **module.exports**. Also, observe the use of the class keyword to create classes in JavaScript. Constructor is another keyword used in this class. By definition, the constructor is the function that is activated when an instance of the class object is created. Write the following code in Person.js

```
class Person {
    constructor(name, age, email) {
      this.name = name;
      this.age = age;
      this.email = email;
    }

    getPersonInfo() {
      return{
        Name: this.name,
        Age: this.age,
        Email: this.email}

    }
  }

  module.exports = Person;
```

Step 3:Now use the functions and object defined in these two files i.e. (StudentInfo.JS, Person.JS**) into file index.js .**

we use "require" keyword to import a module in nodejs JS.

notice that in the require statement, the module name(which is simply the name of the js file where this module is defined) is prefixed with ./, as it's a local file. Also note that there's no need to add the file extension. In our case, the names of modules are StudenInfo and Person, as these are the names we have given the two JS files.

The updated index.js is given below, with the added code highlighted with a blue background.

index.js

```
function EmployeeInfo(name, Salary)
{
console.log("Welcome " + name + " Your monthly Salary is " +
Salary);
}

// Display current date and time
const currentDateTime = new Date();

console.log("This is my first program");
console.log("Programe run on : "
+  currentDateTime.toLocaleString());

var EmpName = "John";
var EmpSalary = 50000;

// Calling the function
EmployeeInfo(EmpName, EmpSalary);
// Add this for Second Exercise
const EmpSkills= (skills,Name,LabGrp)=> {
  console.log(" Student ID"+ Name+ "Expert in "+ skills)
  console.log("Lab Group Name",LabGrp)
}
EmpSkills("java","Replace this with your Student ID ","Replace
with your Lab Grp")

// code for third Exercise
const student= require('./StudentInfo')
const person = require('./Person')
// because getName is the function so we use ()
console.log("Student Name:" +student.getName())
console.log(student.Location())
console.log(student.dob)
// because dob is a variable so we do nt use ()
console.log(student.Studentgrade())
console.log("grade is "+student.Studentgrade(55) )
person1= new person("Jim","USA","myemail@gmail.com")
console.log("using Person Module",person1.getPersonInfo())
console.log("Programe ended")
```

Step 4 :Now Run it you should see following output in the console

However plese provide your infromation
In this line before running the code

```
EmpSkills("java","Replace this with your Student ID ","Replace with your Lab Grp")
```

Runthe programme using node index.js

Exercise 4: NodeJS core Modules

Core module used : os
Objectives: to display the laptop information on the terminal using os Module.
NodeJS functions used :require (all small letters)
Methods of core Module os used are :

Core Module used	Method of the module used	Any other Module used for this method	Syntax used
os	tmpdir	No	os.tmpdir()
os	platform	No	os.platform()
os	release	No	os.release()
os	uptime	No	os.uptime()
os	totalmem	No	os.totalmem()
os	freemem	No	os.freemem()
os	cpus	util	util.inspect(os.cpus())
os	networkInterfaces	util	util.inspect(os.networkInterfaces())

Note :console.log is used to display the text message on screen. Also it is case sensitive and all letters are small letters
A text message is always enclosed with two “” for example
Console.log(“hello”) will display a hello on screen. If you want to concatenate a textmessage with the value of variable or a function then you use a “+” Sign. For example in third line of the code below console.log("temporary directory"+ os.tmpdir()) the text message is “temporary directory ” and after + sign tmpdir() is method that will return the

name of the temporary directory and will concatenate it with the text message. All set to start your first code. It will use os core module from the nodejs and will display information about your laptop such as userinfo,hostname , free memory operating system etc.

Add following lines in your index.js files and observe the output in terminal window

```
os=require("os")
const util=require('util')
console.log("temporary directory"+ os.tmpdir() )
console.log("hostname: "+ os.hostname())
console.log("OS : " + os.platform() +"release:"+ os.release())
console.log("Uptime"+  (os.uptime())/3600 +"  hours")
console.log("userInfo" + util.inspect(os.userInfo()))
console.log("Memory "+ os.totalmem()/1000000000 + "Giga byte")
console.log(" free: "+os.freemem()/1000000000 + "Giga byte")
console.log("CPU "+  util.inspect(os.cpus()))
console.log("Network"+ util.inspect(os.networkInterfaces()))
console.log("programe end")
```

node index.js to run the program

For your portfolio, write a report covering today's lab focusing on the following :

Submit the code for the completed Exercises i.e.:

i) Index.js
ii) Person.js
iii) Employeeinfo.js
iv) Exercise 4.js

Portfolio Activity: Learning Development and Reflection Log

Create a new file called **PortfolioTask.js** and write a program that:

1. Accepts and displays:
    - Your Full Name
    - Your Student ID
    - Your Lab Group Number
2. Displays the current **date and time** using JavaScript.
3. Creates an arrow function that displays your favourite programming language.
4. Calculates a unique value using your Student ID:
    - Add together the last 4 digits of your Student ID.
    - Display the result in the terminal.
5. Take a screenshot showing:

- The complete code
- The terminal output
- The system clock/date visible on your computer

Example Output

Plain Text
Student Name: John Smith
Student ID: U1234567
Lab Group: Tuesday Group A

Current Date and Time:
09/10/2026 15:45:32

Favourite Programming Language: JavaScript

Verification Number: 22

After completing all lab exercises, complete the following reflective learning log.

Exercise	Evidence of Completion (Screenshot Reference)	What I Learned	Challenges or Errors Encountered	How I Solved the Problem	Key Skill Developed
Exercise 1: JavaScript Functions					
Exercise 2: Arrow Functions					
Exercise 3: Local Modules					
Exercise 4: Node.js Core Modules					
PortfolioTask.js					

Evidence Required

Students must provide evidence that all exercises have been executed. Screenshots must show the code, terminal output, and system date/time. **Students must personalise the program output using their own name and student ID and answer the execution verification questions.** Portfolios containing only copied code without evidence of execution and reflection will not meet the portfolio requirements.

FileName Formate:LAbGroup_StudentID_LabtutorName.pdf